

Compte Rendu : E3-TP2 Langages de Script

Module : Systèmes d'Exploitation / Scripting

Sujet : Analyse de l'environnement Shell, script de surveillance système et script réseau Python.

(01) Les Shells installés sur la distribution

Pour connaître la liste des shells valides et installés sur une distribution Linux standard, on consulte le fichier `/etc/shells`.

Commande :

```
cat /etc/shells
```

Résultat typique :

La liste inclut généralement :

- `/bin/sh` (Bourne Shell)
- `/bin/bash` (Bourne Again Shell - le plus courant)
- `/bin/dash` (Debian Almquist Shell)
- `/bin/rbash` (Restricted Bash)

(02) Shell par défaut et Interpréteur du script

Shell par défaut

Pour identifier le shell par défaut attribué à l'utilisateur courant, on consulte la variable d'environnement `$SHELL`.

Commande :

```
echo $SHELL
```

Réponse : Sur notre distribution, le shell par défaut est `/bin/bash`.

Interpréteur du script

En analysant la syntaxe du script fourni (boucles `for`, substitution `$()`, `echo -e`), l'interpréteur déduit est **Bash**.

Emplacement de l'interpréteur

Pour trouver le chemin absolu :
Commande :
which bash

Réponse : /bin/bash.

(03) Analyse des variables et commandes (HOME, du, cut)

La variable HOME

- **Définition** : C'est une variable d'environnement qui contient le chemin absolu vers le répertoire personnel de l'utilisateur courant.
- **Valeur** : /home/leo (dans le contexte de ce TP).

La commande ls \$HOME

L'instruction `ls \$HOME` exécute la commande ls pour lister le contenu du répertoire personnel. Le résultat est utilisé par la boucle for pour traiter chaque dossier/fichier un par un.

La commande de taille (du | cut)

La variable t est calculée via : t=\$(du -hs \$HOME/\$rep | cut -f1).

- **du -hs** : Estime l'espace disque utilisé (-h pour format lisible, -s pour total).
- **cut -f1** : Ne garde que la première colonne (la taille), supprimant le nom du dossier.

Test effectué :

```
du -hs $HOME/Documents | cut -f1  
> 8,0K
```

Le résultat montre une taille de 8 Kilo-octets.

(05) Analyse de la commande de comptage (find | wc)

La variable n est calculée via : n=\$(find \$HOME/\$rep -type f | wc -l).

- **find ... -type f** : Recherche récursivement tous les éléments qui sont strictement des fichiers.
- **wc -l** : Compte le nombre de lignes retournées par find.

Test effectué :

```
find $HOME/Documents -type f | wc -l
```

> 0

Le résultat 0 indique qu'il n'y a aucun fichier dans le dossier Documents (il est vide ou ne contient que des dossiers vides).

(06) Script Bash final commenté

Le script a été édité, commenté et sauvegardé sous le nom script_bash_activite_1.sh.

```
#!/bin/bash

#####
# Nom du script : script_bash_activite_1.sh
# Utilité: Affiche la taille et le nombre de fichiers pour chaque élément de $HOME
#####

# Boucle 'for' qui itère sur chaque élément retourné par la commande `ls $HOME`.
for rep in `ls $HOME`
do
    # Calcul de la taille de l'élément courant ($HOME/$rep).
    # 2>/dev/null : redirige les erreurs (ex: permissions refusées) vers la poubelle.
    t=$(du -hs $HOME/$rep 2>/dev/null | cut -f1)

    # Comptage du nombre de fichiers dans l'élément courant.
    n=$(find $HOME/$rep -type f 2>/dev/null | wc -l)

    # Affichage du résultat formaté.
    echo -e "Répertoire: $rep\t Taille: $t\t Nombre de fichiers: $n"
done
```

(07) Test Bash avec interpréteur

Exécution avec l'interpréteur explicite : bash script_bash_activite_1.sh.

Trace d'exécution partielle :

Répertoire: Bureau	Taille: 2,4M	Nombre de fichiers: 78
Répertoire: chat.jpg	Taille: 0	Nombre de fichiers: 1
Répertoire: Documents	Taille: 8,0K	Nombre de fichiers: 0
Répertoire: wordlists	Taille: 1,3G	Nombre de fichiers: 2987

Conclusion : Le script fonctionne correctement, identifiant les dossiers vides, les fichiers uniques et les répertoires volumineux.

(08) Shebang et permissions Bash

- **Shebang :** Ajout de `#!/bin/bash` en tête de fichier pour spécifier l'interpréteur.
- **Permissions :** Application de `chmod +x script_bash_activite_1.sh` pour rendre le script exécutable.
- **Exécution :** `./script_bash_activite_1.sh`.

(09) Script Python : Calcul de sous-réseaux

Correction du code

Le script fourni comportait des erreurs de syntaxe (chaînes non fermées) et d'indentation. Il a été corrigé et enregistré sous `script_python_activite_2.py`.

Le shebang `#!/usr/bin/env python3` a été ajouté pour assurer la portabilité.

Test et Validation

Le script a été rendu exécutable (`chmod +x script_python_activite_2.py`) et testé avec une adresse de classe C (192.168.1.0/24) et un découpage en 8 sous-réseaux (3 bits supplémentaires).

Trace d'exécution :

```
leo@pc-e002-07:~/Bureau$ ./script_python_activite_2.py
```

```
Entrez l'adresse IPv4 en notation CIDR (par exemple, 192.168.1.0/24): 192.168.1.0/24
```

```
Entrez le nombre de bits supplémentaires du masque (par exemple, 3 pour obtenir 8 sous-réseaux) : 3
```

Informations sur les sous-réseaux :

Sous-réseau 1:

- Adresse de base : 192.168.1.0
- Adresse de diffusion : 192.168.1.31
- Plage d'adresses d'hôtes : 192.168.1.1 à 192.168.1.30

...

Sous-réseau 8:

- Adresse de base : 192.168.1.224
- Adresse de diffusion : 192.168.1.255
- Plage d'adresses d'hôtes : 192.168.1.225 à 192.168.1.254

Analyse :

Le script calcule correctement les bornes des sous-réseaux. Avec 3 bits supplémentaires sur un /24, nous obtenons bien un masque de /27 (32 adresses par sous-réseau), ce qui correspond aux intervalles affichés (0-31, 32-63, etc.).